

# EVENTLENS

## Умный подбор event-подрядчиков с доказательством каждого решения

### Документ

Техническое задание, архитектура, каталог AI-моделей, план реализации, API-контракты, тестирование и сценарий демонстрации. Версия 1.0 от 23.09.2026.

### Базовая идея

Explainable Matchmaker + Constraint Doctor. Система сначала применяет проверяемые ограничения, затем детерминированно ранжирует оставшихся кандидатов и показывает пользователю конкретные доказательства. Если выдача пуста, объясняет, какое условие заблокировало результат.

| Основание    | Что фиксируем                                                                           |
|--------------|-----------------------------------------------------------------------------------------|
| Основное ТЗ  | #79-lite: до 3 карточек, объяснимость, занятость, детерминизм, 3 явных исхода           |
| Данные       | 66 анонимизированных профилей; календарь 23.09-31.12.2026                               |
| Текущий стек | React/Vite + Chakra UI + TanStack Query; FastAPI; отдельный AI service; Docker Compose  |
| Приоритет    | Качество объяснений > честная обработка редких/занятых/пустых категорий > скорость > UI |

### Ключевая архитектурная позиция

LLM не принимает решение о том, кто “лучше”, и не может обойти hard constraints. Порядок формируется обычным кодом + фиксированной semantic-моделью. LLM, если включён, получает только подтверждённый evidence и отвечает исключительно за формулировку текста.

# Содержание

1. Назначение и границы решения
2. Разбор исходного ТЗ и критерии приёмки
3. Анализ датасета
4. Концепция EventLens
5. Функциональное ТЗ
6. Matching pipeline и формула ранжирования
7. Explainability / Evidence Ledger
8. Constraint Doctor и What-if Navigator
9. Архитектура приложения
10. API и структуры данных
11. Каталог моделей и технологий
12. Рекомендуемые конфигурации моделей
13. Этапы разработки
14. Тестирование и оценка качества
15. Demo-сценарий и чек-лист Definition of Done
16. Риски и решения
17. Источники

Приложение А. Пример запроса/ответа

Приложение В. Псевдокод matching-движка

## Статус внешней информации

Каталог моделей сверён по официальным документам и model cards на 23.09.2026. API-модели и алиасы могут меняться; перед финальной интеграцией нужно повторно проверить актуальные IDs и тарифы.

# 1. Назначение и границы решения

EventLens - рекомендательный сервис внутри агрегатора event-подрядчиков Казахстана. Пользователь уже находится внутри каталога своего города; задача сервиса - сократить этот каталог до максимум трёх доказуемо подходящих вариантов, а не расширить его новыми подрядчиками. [S1]

## 1.1. Цель продукта

- Вернуть максимум 3 карточки подрядчиков, которые действительно проходят заданные ограничения.
- Для каждой карточки показать 1-2 предложения с конкретными, проверяемыми причинами появления кандидата.
- Сделать одинаковый запрос воспроизводимым: тот же вход -> тот же порядок.
- Не скрывать неудачу: при нулевом результате объяснить, где именно “сломалась” воронка условий.
- Сохранить работу базового сервиса даже без внешнего LLM/API.

## 1.2. Не входит в MVP

- Бронирование, заявки и уведомления подрядчикам.
- Полноценная CRM/личные кабинеты.
- Дообучение нейросети на 66 профилях.
- Сложный дизайнерский UI в ущерб качеству объяснений.
- Автоматическое добавление внешних подрядчиков, которых нет в исходном каталоге.

### Design principle

“Сначала факты, затем AI”. Жёсткие поля решают допуск кандидата; semantic-модель помогает только понять свободный текст description / пользовательское пожелание.

## 2. Разбор исходного ТЗ и критерии приёмки

| ID  | Область                 | Приёмочное правило                                                                | Приоритет |
|-----|-------------------------|-----------------------------------------------------------------------------------|-----------|
| R1  | Вход                    | Город, дата, тип мероприятия, категория, бюджет; опционально длительность и язык. | P0        |
| R2  | Выход                   | До 3 карточек: имя, категория, город, цена, персональное объяснение.              | P0        |
| R3  | Занятость               | busy_dates - hard exclusion; занятый подрядчик не показывается.                   | P0        |
| R4  | Мало кандидатов         | Если <3 - показать доступное количество и объяснить, почему меньше.               | P0        |
| R5  | Детерминизм             | Повторный запуск идентичного запроса даёт тот же порядок.                         | P0        |
| R6  | Три исхода              | matched / category_absent_in_city / constraints_failed должны быть различимы.     | P0        |
| R7  | Latency                 | Ориентир живой проверки - ответ до 10 секунд.                                     | P0        |
| R8  | Объяснения              | Карточки нельзя перепутать, если стереть имена; без общих маркетинговых фраз.     | P0        |
| R9  | Демо                    | Плотная категория, редкая категория, пустой запрос; разные даты меняют выдачу.    | P0        |
| R10 | Pipeline explainability | Команда способна объяснить, что происходит внутри.                                | P0        |

Источник требований: хакатон-документ #79-lite. В нём отдельно указано, что ценность находится в объяснении, а не в самой сортировке, а fine-tuning на 66 записях не требуется. [S1]

### 3. Анализ датасета

| Метрика                     | Значение                        | Комментарий                         |
|-----------------------------|---------------------------------|-------------------------------------|
| Профилей                    | 66                              | всего                               |
| Синтетических               | 13                              | должны быть явно помечены           |
| city_imputed                | 8                               | город восстановлен при подготовке   |
| price_imputed               | 18                              | цена восстановлена при подготовке   |
| Алматы                      | 50                              | профилей                            |
| Астана                      | 15                              | профилей                            |
| Зарубежье                   | 1                               | профиль                             |
| Цена min/median/max         | 100 000 / 550 000 / 6 000 000 ₸ | price_from_kzt                      |
| Средняя занятость           | 50.6 из 100 дней                | по busy_dates                       |
| Средняя занятость в декабре | 23.1 из 31 дня                  | декабрь существенно плотнее         |
| max_hours = null            | 9                               | не привязанные к присутствию услуги |

Распределение профилей по категориям (один профиль может иметь несколько категорий)

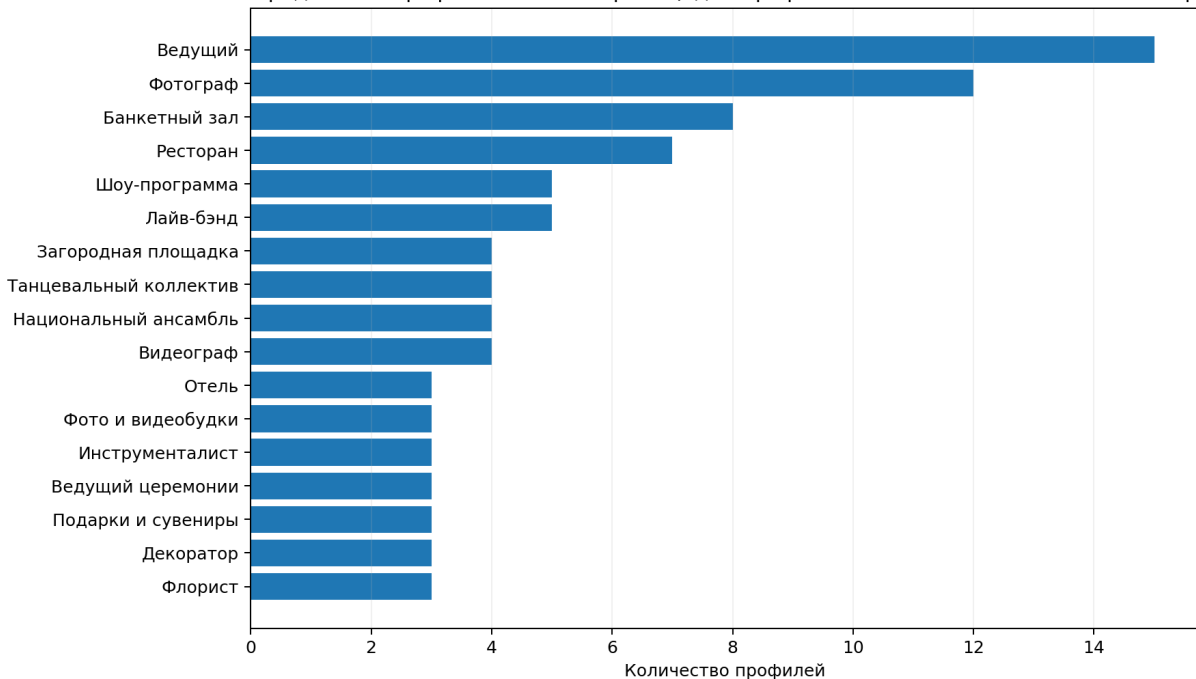


Рисунок 1. Распределение категорий по CSV.

#### 3.1. Выводы для алгоритма

- 66 строк недостаточно для осмысленного fine-tuning. Важнее выбрать готовую multilingual embedding-модель и воспроизводимые правила.
- Плотные категории (“Ведущий”, “Фотограф”, “Банкетный зал”) подходят для демонстрации ranking; категории по 3 профиля - для edge cases.
- Декабрь очень плотный по busy\_dates, поэтому календарь должен быть реальным hard constraint, а не декоративным полем.
- Поля synthetic/city\_imputed/price\_imputed нельзя использовать как скрытое ухудшение score: это происхождение данных, а не качество подрядчика.
- max\_hours = null трактуется как “ограничение длительности неприменимо”, а не как 0 часов.

## 4. Концепция EventLens

### 4.1. Explainable Matchmaker

Основной режим: пользователь задаёт структурированные параметры и, при желании, короткое пожелание по стилю. Система отбрасывает невозможные варианты, затем рассчитывает score для оставшихся и возвращает максимум 3 карточки. В каждой карточке сохраняется evidence ledger - набор фактов, из которых построено объяснение.

| Фактор         | Доказательство                                     | Источник                |
|----------------|----------------------------------------------------|-------------------------|
| Дата           | 18.11.2026 свободен                                | busy_dates              |
| Город          | Алматы совпадает                                   | city                    |
| Формат         | корпоратив поддерживается                          | event_formats           |
| Бюджет         | 1 000 000 ₸ ≤ 1 000 000 ₸                          | price_from_kzt          |
| Язык           | русский поддерживается                             | languages               |
| Длительность   | 6 ч ≤ max_hours                                    | max_hours               |
| Смысл описания | высокое совпадение с "интеллигентно, импровизация" | description + embedding |

### 4.2. Constraint Doctor

Если кандидатов нет, сервис строит воронку: сколько было в категории, сколько осталось после города, даты, формата, бюджета, языка и длительности. Пользователь получает конкретную причину нуля и безопасные варианты изменения условий, рассчитанные только по существующим данным.

### 4.3. Event What-if Navigator (P2)

Расширение для демо: проверить соседние даты в окне календаря и показать количество доступных кандидатов. Это подчёркивает реальную роль busy\_dates и позволяет наглядно показать, почему выдача меняется между датами.

#### Рабочее название

EventLens - "видно не только кого подобрали, но и почему".

## 5. Функциональное ТЗ

| ID    | Функция               | Требование                                                                                                             |
|-------|-----------------------|------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Форма запроса         | Обязательные: city, event_date, event_type, category, budget_kzt. Опциональные: duration_hours, language, preferences. |
| FR-02 | Валидация             | Дата внутри 23.09-31.12.2026; budget > 0; duration > 0; значения category/format/language из meta/options.             |
| FR-03 | Hard filter: category | Кандидат должен содержать выбранную категорию.                                                                         |
| FR-04 | Hard filter: city     | Кандидат должен находиться в выбранном городе. "Зарубежье" не подмешивать без явного выбора.                           |
| FR-05 | Hard filter: date     | event_date не входит в busy_dates.                                                                                     |
| FR-06 | Hard filter: format   | event_type входит в event_formats.                                                                                     |
| FR-07 | Hard filter: budget   | price_from_kzt <= budget_kzt.                                                                                          |
| FR-08 | Hard filter: language | Если язык указан - он обязан входить в languages.                                                                      |
| FR-09 | Hard filter: duration | Если duration задан и max_hours не null - max_hours >= duration. Если max_hours null - фильтр неприменим.              |
| FR-10 | Ranking               | Только прошедшие все hard filters; score фиксированной формулой, затем стабильный tie-break.                           |
| FR-11 | Top K                 | Вернуть min(3, eligible_count).                                                                                        |
| FR-12 | Evidence              | Для каждого кандидата сформировать machine-readable evidence и score_breakdown.                                        |
| FR-13 | Explanation           | 1-2 предложения только из evidence; никакой неподтверждённой информации.                                               |
| FR-14 | Diagnostics           | Всегда возвращать counts по этапам фильтра; при нуле - primary blockers и возможные relaxations.                       |
| FR-15 | Fallback              | Если AI service недоступен, matching работает на TF-IDF/lexical + templates.                                           |
| FR-16 | Determinism           | Фиксировать model version, scoring weights, preprocessing; стабильный tie-break по цене/id.                            |
| FR-17 | Synthetic label       | synthetic=true отображается в debug/demo-режиме.                                                                       |
| FR-18 | Observability         | Логировать query_hash, timings, model version и этап, на котором отсеивались кандидаты; без секретов/API keys.         |

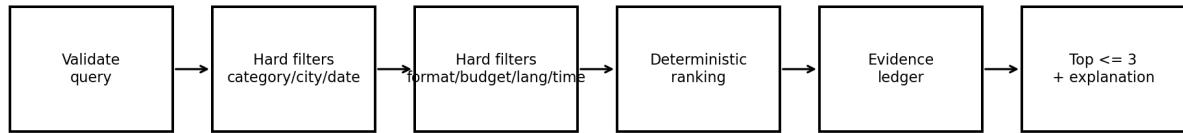
### 5.1. Нефункциональные требования

| ID     | Область         | Правило                                                                        |
|--------|-----------------|--------------------------------------------------------------------------------|
| NFR-01 | Performance     | p95 end-to-end < 10 сек; целевой локальный путь без LLM - существенно быстрее. |
| NFR-02 | Reproducibility | Один docker compose поднимает frontend/backend/ai; версии зависимостей         |

| ID     | Область              | Правило                                                             |
|--------|----------------------|---------------------------------------------------------------------|
|        |                      | закреплены.                                                         |
| NFR-03 | Graceful degradation | AI service timeout не превращается в 500: включается fallback.      |
| NFR-04 | Explainability       | Каждый текстовый тезис трассируется к source_field / evidence code. |
| NFR-05 | Security             | API keys только server-side env; frontend их не получает.           |
| NFR-06 | Testability          | Rule engine и ranker - чистые функции/сервисы с unit tests.         |



## 6. Matching pipeline и формула ранжирования



Если кандидатов 0: Constraint Doctor строит воронку ограничений и причину пустой выдачи.

AI-сервис не меняет hard constraints; при недоступности semantic-модели включается lexical fallback.

Рисунок 2. Последовательность обработки одного запроса.

### 6.1. Жёсткие ограничения

1. Выбрать профили, содержащие category.
2. Оставить только city.
3. Исключить всех, у кого event\_date входит в busy\_dates.
4. Оставить тех, кто берёт event\_type.
5. Оставить price\_from\_kzt <= budget\_kzt.
6. Если language задан - оставить только профили с этим языком.
7. Если duration\_hours задан и max\_hours указан - потребовать max\_hours >= duration\_hours.

### 6.2. Канонический semantic query

Semantic query строится детерминированно из структурированных параметров плюс optional preferences, например: “категория: ведущий; событие: корпоратив; город: Алматы; язык: русский; длительность: 6 часов; пожелания: интеллигентная подача, импровизация”. Это уменьшает зависимость от формулировки UI.

### 6.3. Рекомендуемая формула score

#### Формула при наличии duration

$$\text{score} = 0.50 * \text{semantic\_similarity} + 0.20 * \text{lexical\_similarity} + 0.20 * \text{budget\_fit} + 0.10 * \text{duration\_fit}$$

Если duration не задан или max\_hours неприменим для всей категории: вес duration перераспределяется в semantic\_similarity -> 0.60 semantic + 0.20 lexical + 0.20 budget.

| Компонент           | Расчёт                                                                  | Назначение                                |
|---------------------|-------------------------------------------------------------------------|-------------------------------------------|
| semantic_similarity | cosine(query_embedding, description_embedding), нормализованный в [0,1] | Смысл свободного текста                   |
| lexical_similarity  | TF-IDF cosine или BM25, нормализованный в [0,1]                         | Прямые слова/фразы; fallback              |
| budget_fit          | 1 - price_from_kzt / budget_kzt, ограниченный [0,1]                     | Запас по бюджету; не заменяет hard budget |
| duration_fit        | min((max_hours-duration)/max(duration,1), 1), либо neutral              | Запас по длительности                     |

#### Важно

Весовые коэффициенты - стартовая спецификация, а не “истина”. Для хакатона их следует заморозить после

небольшого ручного eval-набора и не менять между демонстрационными запусками.

## 6.4. Стабильный tie-break

Компоненты score округляются, например, до 6 знаков. Сортировка: (-final\_score, price\_from\_kzt, id). При одинаковых данных результат всегда идентичен. Нельзя использовать random, текущую дату, LLM-score с temperature или непинованный "latest" alias в решающем ranking.

## 7. Explainability: Evidence Ledger

Evidence Ledger - центральная структура решения. Сначала backend создаёт машиночитаемые факты, и только потом renderer превращает их в пользовательский текст. Благодаря этому объяснение можно тестировать и доказывать.

| Code           | Фактор       | Условие                               | Тип                  |
|----------------|--------------|---------------------------------------|----------------------|
| date_available | Дата         | event_date отсутствует в busy_dates   | hard                 |
| city_match     | Город        | profile.city == query.city            | hard                 |
| category_match | Категория    | category входит в categories          | hard                 |
| format_match   | Формат       | event_type входит в event_formats     | hard                 |
| budget_match   | Бюджет       | price_from_kzt <= budget_kzt          | hard + score         |
| language_match | Язык         | language входит в languages           | hard, если указан    |
| duration_match | Длительность | max_hours >= duration                 | hard, если применимо |
| semantic_match | Смысл        | similarity + top matched phrases/tags | soft                 |

### 7.1. Генерация объяснения

- MVP: детерминированные шаблоны. Это лучший вариант для reproducibility и защиты от hallucination.
- AI wording (опционально): LLM получает JSON evidence и инструкцию “не добавлять фактов; 1-2 предложения; числа не изменять”.
- Если LLM timeout/ошибка/schema violation - автоматически вернуть template version.
- LLM-ответ не влияет на score, shortlist и order.
- В debug-панели можно показать вклад score и source\_field каждого доказательства.

## 8. Constraint Doctor и What-if Navigator

### 8.1. Воронка ограничений

| Ключ           | Что означает                     |
|----------------|----------------------------------|
| category_total | Профилей выбранной категории     |
| city           | После города                     |
| date           | После исключения занятых         |
| format         | После формата                    |
| budget         | После бюджета                    |
| language       | После языка (если задан)         |
| duration       | После длительности (если задана) |

Для каждого отсеянного профиля также хранится первая и/или все причины rejection. Это позволяет сказать не “ничего не найдено”, а “в Астане есть 1 флорист, но он занят на выбранную дату”.

### 8.2. Диагностические исходы

| outcome                 | Условие                                  | UI                                                  |
|-------------------------|------------------------------------------|-----------------------------------------------------|
| matched                 | После всех фильтров $\geq 1$             | Вернуть 1-3 кандидата + diagnostics                 |
| category_absent_in_city | После category+city = 0                  | Явно сказать, что в этом городе такой категории нет |
| constraints_failed      | После category+city $> 0$ , но финал = 0 | Назвать блокирующие условия и counts                |

### 8.3. Relaxation suggestions

Constraint Doctor может вычислять безопасные варианты, не подменяя запрос: ближайшая дата с доступностью, минимальный бюджет среди кандидатов, отключение optional language, либо соседний город только как явное действие пользователя. Любое предложение строится пересчётом по тому же каталогу.

### 8.4. What-if Navigator (P2)

- Сканировать, например, +/-7 дней внутри календарного диапазона.
- Для каждой даты считать eligible\_count тем же constraint engine.
- Показать “появились/исчезли” кандидаты с причиной date availability.
- Не хранить отдельную “AI-логику” для календаря - использовать тот же deterministic pipeline.

## 9. Архитектура приложения

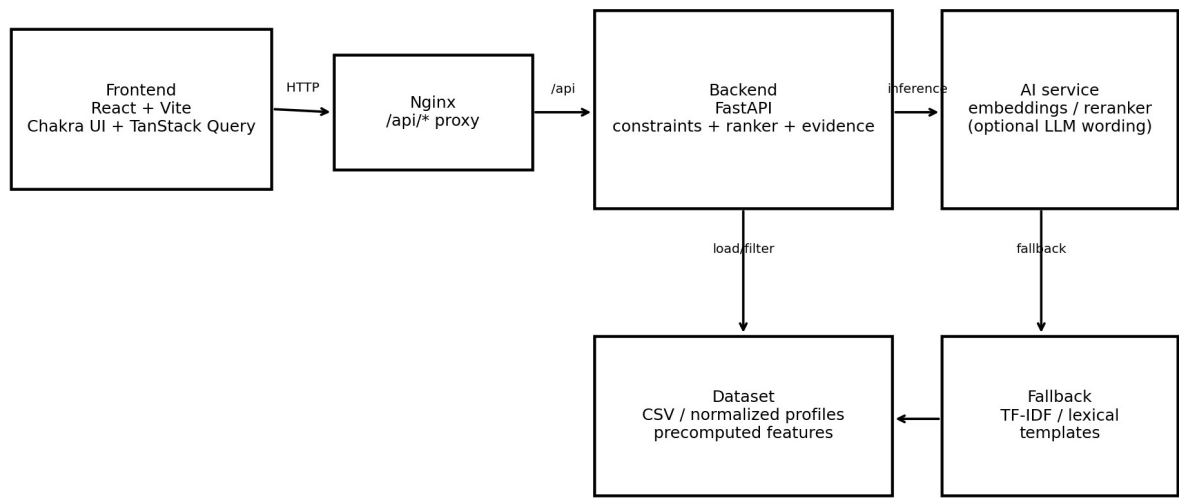


Рисунок 3. Архитектура поверх существующего каркаса репозитория.

Текущий ARCHITECTURE.md уже разделяет frontend (React/Vite/Chakra UI/TanStack Query), FastAPI backend и services/ai; Nginx проксирует /api/\*, а AI service изолирован от backend-образа. EventLens сохраняет эту схему и помещает deterministic business rules в backend, а тяжёлые model dependencies - в services/ai. [S2]

| Компонент     | Ответственность                                                                                               | Технологии                                          |
|---------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| frontend      | Форма запроса, карточки top-3, evidence chips, diagnostics funnel, what-if calendar                           | React/Vite, Chakra UI, TanStack Query               |
| backend       | Pydantic schemas, validation, hard filters, deterministic scoring orchestration, evidence templates, outcomes | FastAPI                                             |
| services/ai   | Embeddings, optional reranker, optional LLM rephrasing                                                        | sentence-transformers/Transformers или внешние APIs |
| dataset layer | CSV loader, normalization, precomputed description embeddings/features                                        | pandas/Pydantic; без vector DB для 66 профилей      |
| infra         | Reverse proxy, healthchecks, env, model cache volume                                                          | Docker Compose + Nginx                              |

### 9.1. Почему vector database не нужен для MVP

При 66 профилях embedding matrix помещается в памяти, cosine similarity считается мгновенно. Qdrant/pgvector можно добавить как масштабируемое продолжение, но на хакатоне они повышают операционную сложность без заметной пользы.

### 9.2. Fault tolerance

- Backend всегда содержит lexical ranker и template explanations.
- AI service имеет /health и ограниченный timeout.
- При старте AI service может заранее вычислить embeddings для 66 descriptions и держать их в RAM/cache.
- Если внешний embedding API выбран, профили индексируются один раз и кэшируются; на пользовательский запрос отправляется только query embedding.

## 10. API и структуры данных

### 10.1. Public endpoints

| Endpoint                     | Назначение                                       |
|------------------------------|--------------------------------------------------|
| POST /api/v1/match           | Основной подбор + diagnostics                    |
| GET /api/v1/meta/options     | Города, категории, форматы, языки, диапазон дат  |
| GET /api/v1/health           | Состояние backend и режим fallback               |
| GET /api/v1/debug/model-info | Версии ranking config/model; можно скрыть в prod |

### 10.2. Internal AI endpoints

| Endpoint              | Назначение                                          |
|-----------------------|-----------------------------------------------------|
| POST /ai/v1/embed     | Embedding списка строк/query                        |
| POST /ai/v1/rerank    | Опциональный cross-encoder rerank shortlist         |
| POST /ai/v1/verbalize | Опциональная формулировка текста строго из evidence |
| GET /ai/health        | Model readiness + version                           |

### 10.3. Match response contract

| Поле                    | Смысл                                                      |
|-------------------------|------------------------------------------------------------|
| outcome                 | matched   category_absent_in_city   constraints_failed     |
| query_hash              | Хэш нормализованного запроса для воспроизводимости         |
| candidates[]            | 0..3 карточки                                              |
| score / score_breakdown | Числа для debug/демо; можно скрыть в обычном UI            |
| evidence[]              | code, label, source_field, value                           |
| explanation             | 1-2 предложения                                            |
| diagnostics.counts      | Воронка кандидатов                                         |
| diagnostics.blockers    | Причины нуля/неполной тройки                               |
| model_info              | ranking_version, semantic_model, fallback_used, timings_ms |

## 11. Каталог моделей и технологий

Нижне - практическая база вариантов, релевантных именно этому проекту. Она разделена по роли в pipeline: lexical baseline, embeddings, rerankers и генерация текста. Сравнивать embedding-модель и LLM как взаимозаменяемые компоненты некорректно: у них разные задачи.

### Критерии выбора

Multilingual (особенно русский/казахский), стабильность, лицензия, простота Docker-развёртывания, latency, возможность pin версии, отсутствие необходимости fine-tuning, работа с короткими русскоязычными descriptions.

### 11.1. Без нейросети: обязательный baseline

| Вариант                  | Где      | Плюсы                                                                  | Минусы                                 | Для EventLens            |
|--------------------------|----------|------------------------------------------------------------------------|----------------------------------------|--------------------------|
| TF-IDF cosine            | Локально | Очень прост, детерминирован, быстрый, объяснимый                       | Не понимает перефразирование/семантику | Да - fallback и baseline |
| BM25                     | Локально | Сильный lexical retrieval, устойчив к редким словам                    | Тоже не semantic                       | Да, альтернатива TF-IDF  |
| Rule-based keywords/tags | Локально | Максимально объяснимо; помогает извлечь "импровизация", "бизнес-форум" | Нужно поддерживать словарь             | Опционально              |

### 11.2. Open-source / local embedding models

| Модель                                  | Факты                                        | Сильная сторона                                                 | Вердикт для проекта                                                |
|-----------------------------------------|----------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|
| paraphrase-multilingual-MiniLM-L12-v2   | 50 языков; 384D; Apache-2.0                  | Лёгкий CPU baseline                                             | Очень удобный для MVP; качество проверить на RU/KZ                 |
| Alibaba-NLP/gte-multilingual-base       | 75 языков; ~0.3B; Apache-2.0                 | Сильный компактный multilingual encoder                         | Хороший кандидат для CPU/среднего контейнера                       |
| intfloat/multilingual-e5-large          | 94 языка; MIT                                | Зрелый multilingual retrieval                                   | Вес модели заметно выше MiniLM                                     |
| intfloat/multilingual-e5-large-instruct | 94 языка; MIT; instruction-aware             | Можно задавать retrieval instruction                            | Тяжелее; для 66 строк может быть overkill                          |
| BAAI/bge-m3                             | Multilingual; MIT; dense/sparse/multi-vector | Сильный универсальный retrieval                                 | Больше зависимостей/ресурсов                                       |
| Qwen3-Embedding-0.6B                    | 100+ языков; 32K; <=1024D; Apache-2.0        | Современный multilingual; настраиваемые dimensions/instructions | Рекомендуемый local AI-кандидат                                    |
| Qwen3-Embedding-4B                      | 100+ языков; 32K; <=2560D; Apache-2.0        | Выше качество                                                   | Требует гораздо больше памяти/GPU; не нужен для 66 строк           |
| Qwen3-Embedding-8B                      | 100+ языков; 32K; <=4096D; Apache-2.0        | Максимум серии                                                  | Хакатонный overkill без GPU                                        |
| jina-embeddings-v4                      | Multilingual, 32K, universal/multimodal      | Современная сильная модель                                      | 3.8B класс - тяжелее; лицензирование/режим использования проверить |

### 11.3. Embeddings as API

| Модель                        | Ключевые свойства                | Плюс                                   | Роль            |
|-------------------------------|----------------------------------|----------------------------------------|-----------------|
| OpenAI text-embedding-3-small | 1536D default; multilingual; API | Очень простой и дешёвый semantic layer | Хороший API-MVP |

| Модель                        | Ключевые свойства                         | Плюс                            | Роль                                                  |
|-------------------------------|-------------------------------------------|---------------------------------|-------------------------------------------------------|
| OpenAI text-embedding-3-large | 3072D default; strongest OpenAI embedding | Больше качества, дороже small   | Использовать только если eval даёт заметный прирост   |
| Voyage voyage-4-lite          | 32K; 1024 default; multilingual           | Latency/cost optimized          | Сильный API-вариант                                   |
| Voyage voyage-4               | 32K; 1024 default; multilingual           | Баланс качества/стоимости       | Сильный API-вариант                                   |
| Voyage voyage-4-large         | 32K; 1024 default; multilingual           | Лучшее качество серии           | Для 66 строк чаще избыточен                           |
| Cohere embed-v4.0             | 128K; 256/512/1024/1536; multilingual     | Гибкие dimensions + multimodal  | Хороший API-вариант                                   |
| Google gemini-embedding-2     | до 8192 input; dimensions до 3072         | Универсальный API embedding     | Хороший вариант, если уже используется Gemini stack   |
| Mistral mistral-embed         | 1024D                                     | Простой text embedding endpoint | Нормальная альтернатива; RU/KZ проверить eval-набором |

## 11.4. Rerankers (опциональный второй этап)

Reranker полезен, если после hard filters остаётся достаточно кандидатов и embedding similarity плохо различает top-3. При 66 профилях он не обязателен: добавлять только после измеримого улучшения ручного eval.

| Модель                                 | Факты                                      | Преимущество                                     | EventLens                                    |
|----------------------------------------|--------------------------------------------|--------------------------------------------------|----------------------------------------------|
| Qwen3-Reranker-0.6B                    | 100+ языков; 32K; Apache-2.0               | Лучший баланс local quality/resource среди серии | P1/P2                                        |
| Qwen3-Reranker-4B                      | 100+ языков; 32K; Apache-2.0               | Выше качество                                    | Overkill без GPU                             |
| Qwen3-Reranker-8B                      | 100+ языков; 32K; Apache-2.0               | Макс. серия                                      | Не рекомендуется для хакатона                |
| BAAI/bge-reranker-v2-m3                | Multilingual; Apache-2.0; ~2.27 GB weights | Зрелый multilingual reranker                     | Хорошая local альтернатива                   |
| Alibaba gte-multilingual-reranker-base | ~0.3B; multilingual family                 | Компактный cross-encoder                         | Хороший local вариант                        |
| mixedbread mxbai-rerank-base-v2        | 109 языков; Apache-2.0                     | Современный multilingual rerank                  | Интересный local вариант                     |
| Cohere rerank-v4.0-fast                | Multilingual API                           | Низкая latency/high throughput                   | Лучший API-rerank для демо                   |
| Cohere rerank-v4.0-pro                 | Multilingual API                           | Более сложные cases/качество                     | Если качество важнее latency                 |
| Jina multilingual reranker v2 base     | Multilingual                               | Компактная модель                                | Проверить лицензию для способа развёртывания |

## 11.5. LLM для формулировки объяснений (не для ranking)

| Модель                       | Факты                                             | Когда использовать                           | Приоритет           |
|------------------------------|---------------------------------------------------|----------------------------------------------|---------------------|
| Без LLM: templates           | 100% deterministic, zero network                  | Лучший MVP и fallback                        | P0 / основной       |
| OpenAI GPT-6 Luna            | Эффективный high-volume model, structured outputs | Хорош для JSON -> короткий текст             | P1 optional         |
| Google Gemini 3.1 Flash-Lite | Stable low-cost model                             | Быстрое переформулирование                   | P1 optional         |
| Google Gemini 3.5 Flash-Lite | Stable, high-throughput                           | Альтернатива 3.1                             | P1 optional         |
| Anthropic Claude Haiku 4.5   | Быстрый API LLM                                   | Короткие аккуратные тексты                   | P1 optional         |
| Mistral Small 4              | Apache-2.0 / API+weights; 256K                    | Сильный generalist, но крупный               | P2 / overkill       |
| Ministral 3 3B               | Apache-2.0; edge-oriented; 256K                   | Локальная генерация на более скромном железе | P1/P2 local wording |



## 11.6. Что НЕ рекомендуется

- Fine-tuning LLM/embedding на 66 профилях: мало данных, высокий риск запоминания и нулевая окупаемость в рамках хакатона.
- LLM-as-a-judge для финального ranking в live path: ухудшает determinism, latency и объяснимость.
- Vector DB только ради "AI-архитектуры": на 66 строках достаточно in-memory matrix.
- Случайная генерация объяснений с temperature > 0 без fallback/validation.
- Использование непинованных latest aliases в компоненте, который влияет на порядок.

## 12. Рекомендуемые конфигурации моделей

| Профиль                   | Стек                                                                 | Плюс                                                      | Когда                                         |
|---------------------------|----------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------|
| A. Надёжный MVP           | TF-IDF + deterministic scoring + templates                           | Ноль внешних зависимостей; полностью reproducible; быстро | Первый рабочий вертикальный срез              |
| B. Рекомендуемый local AI | Qwen3-Embedding-0.6B + TF-IDF + deterministic scoring + templates    | Сильная multilingual semantics, API не нужен              | Основной вариант, если контейнер/железо тянет |
| C. Лёгкий local           | gte-multilingual-base или multilingual MiniLM + TF-IDF + templates   | Проще CPU deployment                                      | Если Qwen 0.6B тяжёл для окружения            |
| D. API-first              | OpenAI text-embedding-3-small или voyage-4-lite + TF-IDF + templates | Минимум ML-инфры                                          | Если доступ к API стабилен                    |
| E. Quality+rerank         | Qwen3-Embedding-0.6B -> Qwen3-Reranker-0.6B -> deterministic blend   | Лучше различает top candidates                            | Только если eval доказывает улучшение         |
| F. Polished wording       | Любая B-D + optional GPT-6 Luna / Gemini Flash-Lite verbalizer       | Более естественный текст                                  | Не включать в decision path                   |

### Рекомендуемый выбор для защиты

Сначала довести до идеала конфигурацию A. Затем добавить B или D под feature flag. Reranker и LLM-verbalizer должны быть улучшениями, а не точками отказа.

### 12.1. Мини-evaluation перед фиксацией модели

8. Собрать 30-50 ручных запросов: плотные/редкие категории, RU/KZ language, разные даты и бюджеты.
9. Для каждого запроса вручную отметить релевантность surviving candidates после hard filters.
10. Сравнить TF-IDF, MiniLM/gte, Qwen3-Embedding-0.6B и 1 API-вариант по pairwise предпочтениям/NDCG только там, где есть labels.
11. Выбрать самую простую модель, которая даёт устойчивое улучшение. Зафиксировать model ID/revision и preprocessing.
12. Повторить 20 одинаковых запусков и убедиться, что порядок полностью стабилен.

## 13. Этапы разработки

| #  | Prio | Этап                 | Что делаем                                                             | Выход                          |
|----|------|----------------------|------------------------------------------------------------------------|--------------------------------|
| 0  | P0   | Freeze requirements  | Зафиксировать schema, 3 outcomes, hard/soft rules, demo queries        | spec.md + config version       |
| 1  | P0   | Data layer           | CSV loader, parse pipe-fields, dates, null semantics, Pydantic Profile | unit tests данных              |
| 2  | P0   | Constraint Engine    | Последовательные filters + rejection ledger + funnel counts            | 0 hard-constraint violations   |
| 3  | P0   | Baseline Ranker      | TF-IDF + budget/duration fit + stable tie-break                        | повторяемый top-3              |
| 4  | P0   | Evidence/Explanation | Evidence codes + deterministic templates                               | у каждого тезиса source_field  |
| 5  | P0   | FastAPI contract     | /match, /meta/options, /health, error schema                           | integration tests              |
| 6  | P0   | Frontend flow        | Form -> states -> result cards -> diagnostics                          | 3 outcomes различимы           |
| 7  | P1   | AI semantic layer    | AI service + chosen embedding model + caching + feature flag           | semantic similarity + fallback |
| 8  | P1   | Constraint Doctor    | blockers, counts, relaxation calculations                              | читаемый no-result             |
| 9  | P2   | What-if Navigator    | date scan and delta explanation                                        | calendar demo                  |
| 10 | P0   | Quality gate         | determinism, latency, unique explanations, failure modes               | acceptance suite               |
| 11 | P0   | README/repro/demo    | docker compose, env.example, architecture, demo script                 | жюри запускает и понимает      |

### 13.1. Рекомендуемое распределение по слоям

| Модуль                            | Ответственность                     |
|-----------------------------------|-------------------------------------|
| backend/app/domain/constraints.py | Чистые hard-filter функции          |
| backend/app/domain/ranking.py     | Scoring + tie-break                 |
| backend/app/domain/evidence.py    | Evidence ledger + templates         |
| backend/app/domain/diagnostics.py | Constraint Doctor                   |
| backend/app/api/match.py          | HTTP orchestration                  |
| services/ai/app/embeddings.py     | Semantic model/API adapter          |
| services/ai/app/rerank.py         | Optional reranker                   |
| frontend/src/features/match/*     | Form/results/diagnostics components |

## 14. Тестирование и оценка качества

| Метрика                  | Gate                       | Проверка                                                                                   |
|--------------------------|----------------------------|--------------------------------------------------------------------------------------------|
| Hard constraint accuracy | 100%                       | Ни один returned candidate не нарушает city/date/category/format/budget/language/duration. |
| Top-K                    | <=3                        | Никогда не возвращать 4+.                                                                  |
| Determinism              | 100% same order            | Идентичный normalized request + versions -> идентичный order.                              |
| Busy-date exclusion      | 100%                       | Проверить каждую returned карточку по busy_dates.                                          |
| Outcome correctness      | 100% на acceptance suite   | matched / absent / failed.                                                                 |
| Evidence factuality      | 100%                       | Каждый evidence item пересчитывается из source field.                                      |
| Explanation groundedness | 100% на demo set           | Нет фактов вне evidence.                                                                   |
| Distinctiveness          | ручная проверка + snapshot | Объяснения top-3 не взаимозаменяемы.                                                       |
| Latency                  | <10s по DoD                | Замер end-to-end; отдельно timings filters/embedding/LLM.                                  |
| Fallback                 | 100% success               | AI service unavailable -> valid lexical response.                                          |

### 14.1. Unit tests

- Дата из busy\_dates всегда исключает профиль.
- price\_from\_kzt == budget\_kzt проходит.
- max\_hours == duration проходит.
- max\_hours null не блокирует флориста/декоратора по длительности.
- language filter не применяется, если language отсутствует.
- Многокатегорийный профиль корректно находится по каждой категории.
- Tie score сортируется price asc, затем id asc.

### 14.2. Integration / contract tests

- JSON schema request/response.
- AI timeout -> fallback\_used=true и HTTP 200 с корректной выдачей.
- Один запрос повторить 20 раз: JSON candidates[].id одинаковы.
- Изменить только дату: проверить, что busy profile исчез/появился и diagnostics это отражает.

## 15. Демо-сценарий и чек-лист Definition of Done

Ниже - конкретные сценарии, проверенные по загруженному CSV. Перед выступлением их следует прогнать через финальную реализацию и зафиксировать скриншоты/expected IDs в automated demo test.

| Сценарий   | Запрос                                                                       | Что демонстрирует                                                                                    |
|------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| А: плотная | Алматы; 18.11.2026; Ведущий; корпоратив; бюджет 1 000 000 ₸; русский; 6 ч    | В исходном CSV этим hard constraints соответствуют несколько профилей, поэтому видна работа ranking. |
| В: редкая  | Алматы; 15.10.2026; Флорист; свадьба; бюджет 300 000 ₸; русский              | В редкой категории ожидается меньше трёх результатов - UI должен честно объяснить неполную тройку.   |
| С: пустая  | Астана; 20.12.2026; Флорист; свадьба/корпоратив; бюджет >=300 000 ₸; русский | В Астане флорист есть, но профиль занят 20.12; Constraint Doctor должен показать date blocker.       |

### 15.1. Live demo sequence

13. Показать запрос А и раскрыть “Почему” у 3 карточек: дата, бюджет, язык, длительность, semantic evidence.
14. Повторить А без изменения параметров - порядок IDs должен быть тем же.
15. Поменять только дату на день, где один из top candidates занят - показать изменение списка и причину.
16. Показать В - вернуть столько, сколько есть, без выдуманного третьего профиля.
17. Показать С - не пустой экран, а funnel и “кандидат есть, но занят на дату”.
18. На 20-30 секунд открыть debug/model-info: rule engine -> semantic similarity -> deterministic score -> evidence -> optional wording.
19. Если есть возможность - отключить AI service и показать, что TF-IDF fallback всё равно даёт валидную выдачу.

### 15.2. DoD checklist

- ☐ Ответ <= 10 сек
- ☐ Топ <= 3
- ☐ Нет занятого подрядчика
- ☐ Explanation конкретное и различимое
- ☐ Повторный запрос = тот же порядок
- ☐ Две даты могут дать разный результат
- ☐ 3 outcomes различимы
- ☐ Есть плотный, редкий и пустой demo
- ☐ Пустой результат объяснён словами
- ☐ README позволяет запустить проект
- ☐ Команда объясняет pipeline

## 16. Риски и решения

| Риск                       | Проявление                                             | Митигация                                                                                                    |
|----------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| LLM hallucination          | Объяснение добавляет несуществующий факт               | Templates by default; evidence-only schema; validate output; fallback.                                       |
| Нестабильный ranking       | API/model version меняется                             | Pin model/revision; fixed weights; stable tie-break; cache profile embeddings.                               |
| Слабый казахский retrieval | Multilingual model хуже на конкретных KZ формулировках | Мини-eval на RU/KZ; hybrid lexical+semantic; при необходимости выбрать Qwen/BGE/API.                         |
| API outage/лимиты          | Демо зависнет                                          | Local/lexical fallback; short timeout; health indicator.                                                     |
| Декабрь даёт почти пусто   | Сезонность воспринимают как баг                        | Constraint Doctor + what-if date counts; объяснить dataset seasonality.                                      |
| Synthetic/imputed fields   | Жюри думает, что система скрывает происхождение        | Demo/debug label; не использовать происхождение как ranking quality signal.                                  |
| Overengineering            | Vector DB/agents съедают время                         | PO сначала; AI под feature flag; не добавлять компонент без измеримого value.                                |
| "AI" не виден              | Решение выглядит как фильтры                           | Показать semantic similarity, сравнение lexical vs embedding и evidence, но оставить decision deterministic. |

## 17. Источники

### Внутренние источники проекта

- [S1] "HackAlem AI\_ Хакатон-задача\_ умный подбор подрядчиков.docx" - официальное ТЗ #79-lite.
- [S2] ARCHITECTURE.md - существующая структура React/Vite/Chakra UI/TanStack Query + FastAPI + services/ai + Docker Compose.
- [S3] hackathon dataset anonymized .csv - фактические 66 профилей, использованные для статистики и демо-сценариев.
- [S4] hackathon dataset preview.html - визуальный preview 66 профилей и признаков.

### Внешние источники по моделям (проверены 23.09.2026)

- [W1] OpenAI Embeddings: <https://developers.openai.com/api/docs/guides/embeddings>
- [W2] OpenAI model catalog: <https://developers.openai.com/api/docs/models>
- [W3] Google Gemini model catalog: <https://ai.google.dev/gemini-api/docs/models>
- [W4] Google Gemini Embedding 2: <https://ai.google.dev/gemini-api/docs/models/gemini-embedding-2>
- [W5] Cohere Embed: <https://docs.cohere.com/docs/cohere-embed>
- [W6] Cohere Rerank: <https://docs.cohere.com/docs/rerank>
- [W7] Voyage Embeddings: <https://docs.voyageai.com/docs/embeddings>
- [W8] Mistral models/embeddings: <https://docs.mistral.ai/models> and [https://docs.mistral.ai/studio/knowledge-rag/embeddings/text\\_embeddings](https://docs.mistral.ai/studio/knowledge-rag/embeddings/text_embeddings)
- [W9] Qwen3 Embedding/Reranker: <https://qwenlm.github.io/blog/qwen3-embedding/>
- [W10] BAAI BGE-M3: <https://huggingface.co/BAAI/bge-m3>
- [W11] BGE reranker v2 m3: <https://huggingface.co/BAAI/bge-reranker-v2-m3>
- [W12] multilingual-e5-large: <https://huggingface.co/intfloat/multilingual-e5-large>
- [W13] GTE multilingual base: <https://huggingface.co/Alibaba-NLP/gte-multilingual-base>
- [W14] Multilingual MiniLM: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- [W15] Mixedbread reranker: <https://huggingface.co/mixedbread-ai/mxbai-rerank-base-v2>
- [W16] Jina Embeddings v4: <https://jina.ai/models/jina-embeddings-v4/>

### Версионность

Для reproducible demo в README хранить точные model IDs/revisions и дату проверки. Не полагаться на "latest" aliases там, где модель влияет на similarity/ranking.

## Приложение А. Пример API запроса и ответа

### Пример request

```
POST /api/v1/match
{
  "city": "Алматы",
  "event_date": "2026-11-18",
  "event_type": "корпоратив",
  "category": "Ведущий",
  "budget_kzt": 1000000,
  "duration_hours": 6,
  "language": "русский",
  "preferences": "интеллекгентная подача, импровизация"
}
```

### Пример response (сокращённый)

```
{
  "outcome": "matched",
  "query_hash": "...",
  "candidates": [
    {
      "id": "НК-...",
      "anon_name": "...",
      "price_from_kzt": 650000,
      "score": 0.83,
      "score_breakdown": {
        "semantic": 0.86, "lexical": 0.74,
        "budget": 0.35, "duration": 0.33
      },
      "evidence": [
        {"code": "date_available", "source_field": "busy_dates"},
        {"code": "format_match", "source_field": "event_formats"},
        {"code": "budget_match", "source_field": "price_from_kzt"}
      ],
      "explanation": "..."
    }
  ],
  "diagnostics": {
    "counts": {"category": 15, "city": "...", "date": "...", "format": "...", "budget": "...}
  },
  "model_info": {
    "ranking_version": "v1",
    "semantic_model": "Qwen/Qwen3-Embedding-0.6B",
    "fallback_used": false
  }
}
```



## Приложение В. Псевдокод matching-движка

```

match(query):
    q = validate_and_normalize(query)
    profiles = load_profiles()

    category_pool = filter(category in profile.categories)
    if no profile in category_pool for q.city:
        return outcome=category_absent_in_city

    ledger = ConstraintLedger(category_pool)
    eligible = ledger.apply(city == q.city)
    eligible = ledger.apply(q.date not in busy_dates)
    eligible = ledger.apply(q.event_type in event_formats)
    eligible = ledger.apply(price_from_kzt <= q.budget_kzt)
    eligible = ledger.apply(optional language)
    eligible = ledger.apply(optional duration; max_hours null => N/A)

    if eligible is empty:
        return outcome=constraints_failed, diagnostics=ledger.summary()

    for profile in eligible:
        semantic = semantic_similarity(q.canonical_text, profile.description)
        lexical = tfidf_similarity(q.canonical_text, profile.description)
        budget = budget_fit(q, profile)
        duration = duration_fit(q, profile)
        profile.score = weighted_sum(...)
        profile.evidence = build_evidence(q, profile, scores)

    ranked = stable_sort(eligible, -score, price_from_kzt, id)
    cards = ranked[:3]
    for card in cards:
        card.explanation = template(card.evidence)
        # optional: LLM verbalizer(evidence), validated; never changes ranking

    return outcome=matched, candidates=cards, diagnostics=ledger.summary()

```

### Финальный принцип

Система должна быть способна объяснить каждый return и каждый reject простым фактом из данных. AI улучшает понимание текста, но не отменяет проверяемые ограничения.